الأكاديمية العربية للعلوم والتكنولوجيا والنقل البحري

Arab Academy for Science, Technology & Maritime Transport

Lecture 6 Handout:

Assembly Language Operations

Arithematic Instructions

ADD

BEFORE EXECUTION

AH	10	AL	10

ADD AX,2020H

AFTER EXECUTION

AH	30	AL	30

1010
+2020
3030

BEFORE EXECUTION

AH	10	AL	10
BH	20	BL	20

ADD AX,BX

AFTER EXECUTION

AH	30	AL	30
BH	20	BL	20

ADC

- E.g. (1). ADC AX,2020H;

BEFORE EXECUTION

CY	1		
AH	10	AL	10

1010
+2020
3030+1=3031

AFTER EXECUTION

AH	30	AL	31

- E.g. (2). ADC AX,BX;

BEFORE EXECUTION

CY	1		
AH	10	AL	10
BH	20	BL	20

AFTER EXECUTION

AH	30	AL	31
BH	20	BL	20

INC

INC AX

BEFORE EXECUTION

AH	10	AL	10

AFTER EXECUTION

AH	10	AL	11

INC [5000H]

BEFORE EXECUTION

5000H	1010

AFTER EXECUTION

5000H	1011

DEC

DEC AX

BEFORE EXECUTION

AH	10	AL	10

AFTER EXECUTION

AH	10	AL	0F

DEC [5000H]

BEFORE EXECUTION

5000H	1010

AFTER EXECUTION

5000H	100F

SUB

SUB AX,1000H

BEFORE EXECUTION

AH	20	AL	00

AFTER EXECUTION

AH	10	AL	00

```
 2000
-1000
=1000
```

SUB AX,BX

BEFORE EXECUTION

AH	20	AL	00
BH	10	BL	00

AFTER EXECUTION

AH	10	AL	00
BH	10	BL	00

SBB

STC
SBB AX,1000H

BEFORE EXECUTION

CF	1

AH	20	AL	20

2020
- 1000
1020-1=1019

AFTER EXECUTION

AH	10	AL	19

STC
SBB AX,BX

BEFORE EXECUTION

CF	1

AH	20	AL	20
BH	10	BL	10

AFTER EXECUTION

AH	10	AL	19
BH	10	BL	10

Program to Subtract Two Numbers

- *This example program demonstrates how to subtract 2 from 4 and display the result.*

```asm
.model small
.data
msg db "Subtract 2 from 4$"
.code
main proc
    mov ax, @data
    mov ds, ax
    ; Display message
    mov ah, 09h
    mov dx, offset msg
    int 21h

    ; Newline
    mov ah, 02h
    mov dl, 0ah
    int 21h

    ; Perform subtraction
    mov ax, 4
    mov bx, 2
    sub ax, bx

    ; Exit program
    mov ah, 4ch
    int 21h
main endp
end main
```

OUTPUT:

Subtract 2 from 4
Answer is 2

BCD Number System

- Binary Coded Decimal : use binary to represent 0-9 only.

- Two terms for BCD number:
  - Unpacked BCD
  - Packed BCD

Digit	*BCD*
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001

Unpacked and Packed BCD Numbers

- **Unpacked BCD**: In unpacked BCD (Binary-Coded Decimal), a single byte is used to store one BCD digit, with the higher nibble (4 bits) typically set to 0.
  - For example:
    - `0000 0101` represents the unpacked BCD for 5.
    - `0000 1001` represents the unpacked BCD for 9.
- **Packed BCD**: In packed BCD, a single byte holds two BCD digits, with each nibble representing a separate digit.
  - For example:
    - `0101 1001` represents the packed BCD for 59.
  - This format is twice as efficient in storing decimal data compared to unpacked BCD, as it uses both nibbles of a byte.

Addition of Packed BCD Numbers

- **Using ADD and ADC**: To add two packed BCD numbers, the `ADD` instruction is used for the lower nibbles (the units place), and the `ADDC` (Add with Carry) instruction is used for the upper nibbles (the tens place).
- **Example Calculation**: To add $17_{10} + 18_{10} = 35_{10}$:
  - In hexadecimal, 17_{10} is `17H` and 18_{10} is `18H`.
  - Adding `17H` and `18H` directly in packed BCD results in `2FH`.

```
  1  7
+ 1  8
  2  F
```

- **Adjusting for Correct BCD Result**: After addition, the result `2FH` is not a valid BCD number because `F` exceeds 9. To correct this, the programmer must add `6` to the lower nibble (`F`) to adjust the result to a valid BCD format.

```
  2  F
+ 0  6
  3  5
```

  - After adding `6` to `2F`, the final result is `35H`, which represents 35_{10}.
- **Final Result**: The correct packed BCD sum is `35H`.

DAA

- Decimal adjust for addition
- DA instruction will add 6 to the lower nibble or higher nibble if needed.

DAA

```
MOV AL, 47H
MOV BL, 55H
ADD AL,BL
DAA

;check CY here
```

```
      4  7
    + 5  5
      9  C
      A  2
CY=1  0  2
```

1. Lower nibble
2. Upper nibble

SBB

- Subtraction with borrow

SBB AX, BX ; AX=AX-BX-CY

```
CLC
MOV AL, 3FH
MOV BL, 23H
SBB AL,BL
```

```
  3 F
- 2 3
-----
  1 C
```

- After the subtraction,
  - CY=0: positive result
  - CY=1: negative result

Multiplication and Division:

Example 1: Unsigned Multiplication

To multiply two unsigned values:

```asm
mov bl, 5
mov al, 3
mul bl ; AX = AL * BL
```

Result: AX holds the result of `AL * BL`.

Example 2: Unsigned Multiplication with 16-bit Registers

```asm
mov bx, 5
mov al, 3
mul bx ; DX:AX = AX * BX
```

Ex.1: Assume that each instruction begins with the following values:

AL = 85H, BL = 35H, AH = 0H

1. MUL BL → *AL* x *BL* = *85H* x *35H* = *1B89H*
   Result: AX = 1B89H

Explanation of Unsigned Multiplication

In this example, multiplying `AL = 85H` (133 in decimal) by `BL = 35H` (53 in decimal) produces a result in `AX = 1B89H` (7049 in decimal).

Signed Multiplication (IMUL)

Example: Signed Multiplication

```asm
mov al, -123 ; AL in 2's complement
mov bl, 53
imul bl ; AX = AL * BL
```

2. IMUL BL → 2's complement of *AL* x *BL* = 2's comp. (85*H*) x 35*H*
   Calculation: *7BH* x *35H* = *1977H*
   Convert to 2's complement: E689H
   Result: AX = 1B89H

In signed multiplication, `IMUL` correctly handles negative values using two's complement representation.

Conversion Instructions

CBW: Convert Byte to Word

```asm
mov al, 6
cbw ; AL to AX, sign-extended
```

Example: Using CBW with a Positive Number

```assembly
MOV BL, 4      ; Load 4 into register BL
MOV AL, 6      ; Load 6 into register AL
CBW            ; Convert AL to AX, sign-extending AL into AH if needed
DIV BL         ; Divide AX by BL, result in AL (quotient), remainder in AH
```

$$\text{DIV BL} \rightarrow \frac{AX}{BL}$$

CWD: Convert Word to Double Word

```asm
mov ax, 6
cwd ; AX to DX:AX, sign-extended
```

Example: sing CWD with a Positive Number

```assembly
MOV BX, 4      ; Load 4 into register BX
MOV AX, 6      ; Load 6 into register AX
CWD            ; Convert AX to DX:AX, sign-extending AX into DX if needed
DIV BX         ; Divide DX:AX by BX, result in AX (quotient), remainder in DX
```

$$\text{DIV BX} = \frac{\text{DX}:\text{AX}}{\text{BX}}$$

Conclusion:

Arithmetic operations in assembly language are essential for performing calculations directly on hardware registers, providing precise control over data manipulation. These operations support both basic and complex computations, handling signed and unsigned values, multi-byte calculations, and adjustments for specialized formats like Binary Coded Decimal (BCD). By operating at a low level, these commands enable efficient processing and allow for fine-grained control over program execution, crucial in performance-critical applications.